

TASK 1: PYTHON AND OOPS

1. Least Required Coins

Write a Python program that accepts an amount in coins and calculates the minimum number of coins needed to make up that amount using the following denominations:

- coins = [25, 10, 5, 2, 1]
- Enter the amount: 87
- Output: THE MINIMUM COINS NEEDED ARE: 6

2. Recursive Number Operations

Write a Python program with recursive functions to perform the following operations on a given integer:

- factorial(n)
- sum_of_digits(n)
- reverse_number(n)
- fibonacci(n) Handle invalid inputs and negative values appropriately.

3. Student Record Management with Error Handling

Write a Python program to manage student records using a dictionary containing Roll number, Name, and Marks. Implement functions to add, search, update marks, and display students, along with exception handling for duplicate roll numbers, missing students, invalid marks, and invalid input types via a menu-driven interface.

4. Stack Implementation

Write a Python program to implement a Stack using a class containing push(x), pop(), peek(), and display() methods using a list (without using built-in append() or pop()). Raise appropriate exceptions for stack overflow, underflow, and invalid input in a menu-driven program.

5. Linked List Implementation

Write a Python program to implement a Singly Linked List using Node and LinkedList classes with methods for insertion, deletion from beginning/end, searching, and displaying. Handle empty list deletions and missing elements with proper error handling and a menu-driven interface.

6. Multi-Role Resource and Appointment Management System

Create a Python program for an institutional system (such as a library or hospital) that manages users, resources, and scheduling:

- a) Create an abstract base class Person (or LibraryMember) with core attributes like ID, name, and age, along with abstract methods for system actions.
- b) Derive specialized subclasses (StudentMember, FacultyMember, Doctor, and Patient) with suitable additional attributes and method overriding.
- c) Create companion classes like Book or Appointment containing specific item details, dates, and times using composition.
- d) Implement encapsulation to protect resource availability and status attributes.
- e) Enforce business logic to prevent conflicting operations, such as multiple overlapping appointments for the same doctor or issuing unavailable books.
- f) Raise custom exceptions for invalid IDs, unavailable slots/resources, and invalid inputs, providing a menu-driven interface.

7. Commercial Asset Rental and E-Commerce Transaction System

Create a Python program for a commercial platform that handles physical inventory, rentals, shopping carts, and polymorphic payments:

- a) Create base/abstract classes such as Vehicle (with an abstract method calculateRent()) and Product (with attributes like ID, name, price, and stock).
- b) Create specialized subclasses derived from vehicles (Car, Bike, and LuxuryCar) and override methods like calculateRent(days).
- c) Create an abstract class Payment with makePayment(), and derive subclasses like CardPayment and UPIPayment to demonstrate polymorphism.
- d) Create container classes like Customer and ShoppingCart using composition.
- e) Implement encapsulation for pricing, stock levels, and transaction states.
- f) Raise explicit exceptions for invalid products/vehicles, insufficient stock, invalid rental durations, and failed payments via a complete menu-driven program.